Studio Nx

JS 289 — 오디오 클립 재생

2026-10-19

JavaScript 289 — 오디오 클립 재생 (`audio/playback.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/playback.js`
작업	S1-11a 재생 오디오 그래프: 타임라인의 오디오 클립을 실제 소리로 재생 (FR-205)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 288 믹서, 다음 290 재생 세션

1. 하는 일

오디오 클립을 "언제, 샘플의 어디부터, 얼마나" 울릴지 계산하고, 그 계산대로 `AudioBufferSourceNode`를 예약합니다. 계산은 순수 함수이고, 예약은 `ClipPlayer`가 합니다.

```
클립 { start_tick, length_tick, offset_tick, gain_db, is_looped, pitch_semitones }
        │
        ├─ clipsStartingIn  이번 단계에서 새로 시작하는 클립을 고릅니다
        ├─ clipsCovering    재생 위치가 한가운데 걸친 클립을 고릅니다 (중간부터 재생)
        ▼
planClipPlayback  when(울릴 시각), offsetSec(샘플 안 시작 위치), durationSec(길이)
        ▼
ClipPlayer.schedule  샘플 버퍼 → 게인(클립 gain) → 트랙 입구
```

2. 언제 쓰나

쓰는 곳	하는 일
`PlaybackSession.start`	재생을 시작할 때 시작 위치에 걸친 클립을 `clipsCovering`으로 골라 중간부터 울립니다.
`PlaybackSession._onStep`	스케줄러가 한 단계(1/16박)를 넘길 때마다 `clipsStartingIn`으로 새로 시작할 클립을 골라 예약합니다.
루프 되감기	루프 끝에서 되감기면 `stopAll(time)`으로 옛 소리를 끊고, 되감긴 위치에 걸친 클립을 다시 시작합니다.

3. 코드

3.1 클립 고르기

```
export function clipsStartingIn(clips, fromTick, toTick) {
  return (clips || []).filter(function (c) {
    return c.start_tick >= fromTick && c.start_tick < toTick;
  });
}
```

```
export function clipsCovering(clips, tick) {
  return (clips || []).filter(function (c) {
    return c.start_tick < tick && c.start_tick + c.length_tick > tick;
```

```
  });
}
```

3.2 planClipPlayback

```
export function planClipPlayback(clip, nowTick, nowTime, baseBpm, changes) {
  const startTick = Math.max(clip.start_tick, nowTick);
  const skippedTicks = startTick - clip.start_tick;
  const toSec = function (a, b) {
    return ticksToSecondsWithChanges(b, baseBpm, changes) - ticksToSecondsWithChanges(a, baseBpm, change
s);
  };
  const when = nowTime + toSec(nowTick, startTick);
  const offsetSec = toSec(0, (clip.offset_tick || 0) + skippedTicks);
  const durationSec = toSec(startTick, clip.start_tick + clip.length_tick);
  return { when: when, offsetSec: offsetSec, durationSec: durationSec };
}
```

3.3 ClipPlayer

```
schedule(clip, nowTick, nowTime, baseBpm, changes) {
  if (clip.sample_id == null) return false;
  const buffer = this.getBuffer(clip.sample_id);
  if (!buffer) return false;
  const plan = planClipPlayback(clip, nowTick, nowTime, baseBpm, changes);
  if (!(plan.durationSec > 0)) return false;
  // 샘플보다 뒤에서 시작하면 소리가 없음 (루프가 아닐 때)
  if (!clip.is_looped && plan.offsetSec >= buffer.duration) return false;

  const ctx = this.context;
  const src = ctx.createBufferSource();
  src.buffer = buffer;
  src.loop = !!clip.is_looped;
  const semis = Number(clip.pitch_semitones) || 0;
  if (semis !== 0) src.playbackRate.value = Math.pow(2, semis / 12);
  const gain = ctx.createGain();
  gain.gain.value = dbToGain(clip.gain_db || 0);
  src.connect(gain);
  gain.connect(this.trackInput(clip.track_id));

  const when = Math.max(plan.when, ctx.currentTime);
  src.start(when, plan.offsetSec % (buffer.duration || 1));
  src.stop(when + plan.durationSec);
  const self = this;
  src.onended = function () {
    const i = self.active.indexOf(src);
    if (i >= 0) self.active.splice(i, 1);
  };
  this.active.push(src);
  return true;
}
```

```
stopAll(atTime) {
  const when = atTime != null ? atTime : 0;
```

```
  this.active.slice().forEach(function (src) {
    try { src.stop(when); } catch (e) { /* 이미 끝난 소스 */ }
    if (atTime == null) { try { src.disconnect(); } catch (e) { /* 무시 */ } }
  });
  if (atTime == null) this.active = [];
}
```

4. 최신 문법으로 쓰면

```
// clipsStartingIn 을 화살표 함수와 기본 매개변수로 쓴 모양
export const clipsStartingIn = (clips = [], fromTick, toTick) =>
  clips.filter(({ start_tick }) => start_tick >= fromTick && start_tick < toTick);

// planClipPlayback 의 오프셋 계산을 nullish 병합으로 쓴 모양
const offsetSec = toSec(0, (clip.offset_tick ?? 0) + skippedTicks);
```

전통 문법 (실제 코드)	최신 문법
`(clip.offset_tick || 0)`	`clip.offset_tick ?? 0`
`atTime != null ? atTime : 0`	`atTime ?? 0`
`function (c) { return ...; }`	`(c) => ...`

5. 왜 이렇게

선택	이유
계산을 순수 함수로 분리했습니다	시각과 위치 계산은 틀리기 쉬워서 소리 없이 숫자로 시험합니다. 중간부터 재생, 템포 변경 같은 경우도 같은 함수로 처리합니다.
시작 시각을 `max(when, currentTime)`으로 잡습니다	이미 지난 시각으로 예약하면 소리가 빠지거나 한꺼번에 울립니다. 지금으로 당겨서 울립니다.
재생 중간에 걸친 클립은 샘플을 건너뛰고 시작합니다	재생선이 클립 한가운데일 때 처음부터 울리면 위치가 어긋납니다. `offsetSec`에 이미 흐른 길이를 더합니다.
샘플이 없으면 조용히 건너뜁니다	샘플 내려받기에 실패해도 재생선과 다른 소리는 계속되게 하려는 것입니다.
정지는 `stop` 뒤 `disconnect`입니다	소스 노드는 한 번 쓰면 다시 쓸 수 없어서, 끊고 목록을 비웁니다.
루프 되감기는 시각을 지정해 끊습니다	미리 예약해 둔 소리 중 되감기 이전 구간만 끊어야 하므로, `stopAll(time)`에 되감기 시각을 줍니다.
음정은 `playbackRate`로 바꿉니다	반음 올림은 `2^(반음/12)`배 빠르게 재생하는 것과 같습니다. 길이도 같이 짧아집니다 (워프는 지원하지 않음).

6. 시험

- `docs/handoff/s1-11a_tests/mixer_playback_unit.mjs`는 120BPM에서 960틱이 0.5초인 기준으로 세 경우를 확인합니다. 정상 시작이면 `when`이 그대로이고 오프셋이 0입니다. 중간부터 재생이면 오프셋에 건너뛴 길이와 클립 `offset_tick`이 더해집니다. 아직 먼 클립이면 `when`이 그만큼 뒤입니다.
- `docs/handoff/s1-11a_tests/ui.mjs`는 440Hz 사인파 클립을 재생해 마스터 피크가 0.2에서 0.6 사이인지, 정지하면 무음인지 확인합니다.